

Avbryta och fortsätta slingor

Avbrytning med `break`

I vissa fall kan det vara nödvändigt att i en slingas kropp kunna antingen avsluta hela slingan eller hoppa till slingans start. Det första kan göras med en `break`-sats. Denna avslutar den innersta slingan och fortsätter på raden omedelbart efter slingan. Allmänt ser det ut t.ex.:

```
while ( villkor ) {
    sats1;
    break;
    sats2;
}
sats3;
```

I detta fall kommer `break` att omedelbart avsluta slingan och hoppa till raden direkt efter slingan, d.v.s. raden där `sats3` finns. `sats2` kommer således aldrig att utföras! Ett användningsområde för slingor i iterationssammanhang är för att avsluta en evig slinga:

```
// en evig slinga med hjälp av en for-slinga
for ( ; ; ) {
    // gör något
    ...

    // ett villkor A säger att vi skall avsluta slingan
    if ( A ) {
        break;
    }
}
```

I de flesta fall kan man dock undvika att använda en dylik konstruktion (som ju kan vara svårläslig i vissa fall) genom att göra en normal `while`-slinga med villkoret `A` som slingans villkor. Andra användningsområden för `break` finns i kapitlet om flödeskontroll (se [Kapitel 6](#)).

Fortsättning med `continue`

I motsats till att avsluta en slinga tvärt kanske man vill kunna göra något som säger att "denna iteration är nu klar, testa villkoret på nytt och fortsätt därifrån". Detta görs med hjälp av satsen `continue`. Den allmänna formen för `continue` är:

```
while ( villkor ) {
    sats1;
    continue;
    sats2;
}
sats3;
```

I detta fall kommer `continue` att åstadkomma att exekveringen av slingan fortsätter från `villkor`. Resten av slingan hoppas således helt över, och `sats2` kommer även i detta fall aldrig att utföras! Slingan avslutas dock inte, utan endast `villkor` kan avgöra detta.

Konstruktioner med `continue` kan vara praktiska om man har djupt nästlade villkorssatser och vill kunna "starta om" en slinga som innehåller dessa. Då en `for`-slinga används åstadkommer `continue` att exekveringen fortsätter från slingans *inkrementeringsdel*, medan `while`- eller `do-while`-slingor fortsätter exekveringen från slingans *villkorsdel*. Då man använder `while`-slingor bör man således se upp så man inte i misstag hoppar över den delen av slingan som t.ex. inkrementerar en variabel eftersom det skulle åstadkomma en evig slinga.

[Förutgående](#)
`for`-slingor

[Hem](#)
[Upp](#)

[Nästa](#)
Flödeskontroll